

Tecnatura Superior en Desarrollo de Software

Anexo 05a — Jerarquía de Colecciones Genéricas en C#

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

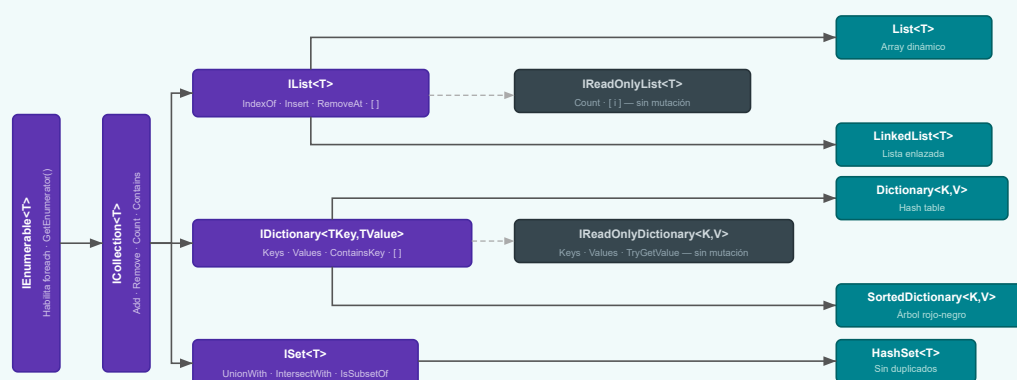
¿Qué es?

El espacio de nombres `System.Collections.Generic` organiza sus tipos en una jerarquía de interfaces y clases concretas. En la cima se encuentra `IEnumerable<T>`, la interfaz mínima que habilita la iteración con `foreach`. A partir de ella, cada nivel agrega responsabilidades: conteo, modificación, acceso por índice, acceso por clave o semántica de conjunto. Las clases concretas implementan una o más de estas interfaces y proveen la gestión de memoria real.

Comprender esta jerarquía permite elegir el tipo más restrictivo adecuado para cada situación: exponer solo lo que el consumidor necesita, sin revelar capacidades internas innecesarias.

Árbol de Jerarquía

■ Interfaz abstracta ■ Clase concreta ■ Interfaz de sólo lectura — Hereda / implementa
■ Implementa (lateral)



Colecciones Principales

`IEnumerable<T>` Úsela cuando solo necesita iterar

Interfaz raíz de toda la jerarquía. Define un único método: `GetEnumerator()`, que devuelve un enumerador capaz de recorrer la secuencia elemento a elemento. Es el tipo mínimo que habilita `foreach` y LINQ. No expone conteo, modificación ni acceso por índice.

Se usa como parámetro de entrada cuando un método solo necesita recorrer datos, sin importar cómo están almacenados internamente.

Cuándo usar en firma:

- Parámetro de entrada de lectura
- Resultado de consulta LINQ
- Métodos de procesamiento genérico

`ICollection<T>` Cuando necesita agregar, quitar y contar

Extiende `IEnumerable<T>` agregando `Count`, `Add`, `Remove`, `Contains` y `Clear`.

Representa una colección mutable de tamaño conocido, sin precisar cómo se accede a cada elemento.

Operaciones clave:

- `Count` $O(1)$
- `Add / Remove`
- `Contains` $O(n)$

`ICollection<T>` → `List<T>` Colección dinámica con acceso por índice

Agrega acceso posicional (`[i]`), `IndexOf`, `Insert` y `RemoveAt` sobre `ICollection<T>`.

`List<T>` implementa esta interfaz con un arreglo interno redimensionable: cuando se supera la capacidad, duplica su tamaño.

Es la colección de propósito general más utilizada en C#. Ver Anexos 03a y 03b para detalle completo.

Complejidades:

- Add $O(1)^*$
- Insert $O(n)$
- [i] $O(1)$
- Remove $O(n)$

IDictionary<TKey,TValue> → Dictionary<TKey,TValue> Búsqueda por clave

Asocia claves únicas a valores. Dictionary<TKey,TValue> implementa esta interfaz mediante una tabla hash: cada clave se convierte en un código hash que determina la posición en memoria. Esto hace que la búsqueda, inserción y eliminación sean prácticamente instantáneas en promedio.

El orden de iteración no está garantizado. Si se necesita orden por clave, usar SortedDictionary<TKey,TValue>.

Operaciones:

- [key] $O(1)^*$
- ContainsKey $O(1)^*$
- TryGetValue $O(1)^*$
- Remove $O(1)^*$

ISet<T> → HashSet<T> · SortedSet<T> Sin duplicados, operaciones de conjunto

Garantiza unicidad de elementos y expone operaciones de teoría de conjuntos: UnionWith, IntersectWith, ExceptWith, IsSubsetOf. HashSet<T> no garantiza orden de iteración. SortedSet<T> mantiene los elementos ordenados usando un árbol rojo-negro.

Operaciones:

- Add $O(1)^*$
- Contains $O(1)^*$
- UnionWith $O(n)$

Queue<T> (FIFO) Procesamiento en orden de llegada

Implementa semántica de cola: el primer elemento ingresado es el primero en salir (*First In, First Out*). Los extremos son: Enqueue agrega al final, Dequeue extrae del frente. Útil para modelar colas de atención, procesamiento de tareas o buffers de eventos.

Operaciones:

- Enqueue $O(1)^*$
- Dequeue $O(1)$
- Peek $O(1)$

Stack<T> (LIFO) Historial, deshacer, recursión iterativa

Implementa semántica de pila: el último elemento ingresado es el primero en salir (*Last In, First Out*). **Push** agrega en la cima, **Pop** extrae de la cima. Modela el historial de operaciones deshacer/rehacer, la evaluación de expresiones o la simulación de recursión de forma iterativa.

Operaciones:

- Push $O(1)^*$
- Pop $O(1)$
- Peek $O(1)$

¿Cuál colección elegir?

La elección depende de la pregunta que responde la estructura:

Pregunta de diseño	Colección adecuada	Razón principal
¿Necesito solo iterar?	<code>IEnumerable<T></code>	Mínima restricción, máxima flexibilidad de entrada
¿Necesito lista ordenada y dinámica?	<code>List<T></code>	Acceso por índice $O(1)$, inserción amortizada $O(1)$
¿Necesito buscar por clave única?	<code>Dictionary<TKey, TValue></code>	Búsqueda $O(1)$ amortizado por hash
¿Necesito elementos únicos?	<code>HashSet<T></code>	Garantía de unicidad, <code>Contains</code> $O(1)$
¿Necesito procesar en orden de llegada?	<code>Queue<T></code>	FIFO semántico, enqueue/dequeue $O(1)$
¿Necesito deshacer/historial?	<code>Stack<T></code>	LIFO semántico, push/pop $O(1)$



Pregunta de diseño	Colección adecuada	Razón principal
¿Necesito tamaño fijo y rendimiento máximo?	<code>T[]</code> (arreglo)	Memoria contigua fija, sin overhead de gestión
Principio de mínima exposición: en firmas públicas, siempre use la interfaz más restrictiva que cubra las necesidades del consumidor. Si el método solo itera, declare <code>IEnumerable<T></code>. Si el consumidor necesita agregar, use <code>ICollection<T></code>. Si necesita índices, use <code>IList<T></code>. Nunca exponga más capacidad de la necesaria.		